

6

Building a Photo Gallery App Using CollectionView and CarouselView

In this chapter, we will build an app that shows photos from the camera roll (photo gallery) of a user's device. The user will also be able to select photos as favorites. We will then look at the different ways to display photos—in carousels and in multi-column grid control. By using the .NET MAUI **CarouselView** control to display a group of images, the user can swipe through them to view each image. To display a large group of images, we will use the .NET MAUI **CollectionView** control and vertical scrolling to allow the user to view all the images. By learning how to use these controls, we will be able to use them in a lot of other cases when we build real-world apps.

The following topics will be covered in this chapter:

- Requesting permissions from the user to access data
- How to import photos from the iOS and Mac Catalyst photo gallery
- How to import photos from the Android photo gallery
- How to import photos from the Windows photo gallery
- How to use **CarouselView** in .NET MAUI
- How to use **CollectionView** in .NET MAUI

Technical requirements

To be able to complete this project, you will need to have Visual Studio for Mac or Windows installed, as well as the necessary .NET MAUI workloads. See *Chapter 1, Introduction to .NET MAUI*, for more details on how to set up your environment.

To build an iOS app using Visual Studio for your PC, you have to have a **Macintosh (Mac)** device connected. If you don't have access to a Mac at all, you can just follow the Android and Windows part of this project.

You can find the full source for the code in this chapter at <https://github.com/PacktPublishing/MAUI-Projects-3rd-Edition/>.

Project overview

Almost all apps visualize collections of data, and in this chapter, we will focus on two of the .NET MAUI controls that can be used to display data collections—**CollectionView** and **CarouselView**. Our app will show the photos that users have on their devices; to do that, we need to create a

photo importer for each platform—one for iOS and Mac Catalyst, one for Windows, and one for Android.

The build time for this project is about 60 minutes.

Building the photo gallery app

This project, like all the rest, is a **File | New | Project...**-style project, which means that we will not be importing any code at all. So, this first section is all about creating the project and setting up the basic project structure.

It's time to start building the app using the following steps. Let's begin!

Creating the new project

The first step is to create a new .NET MAUI project:

1. Open Visual Studio 2022 and select **Create a new project**:

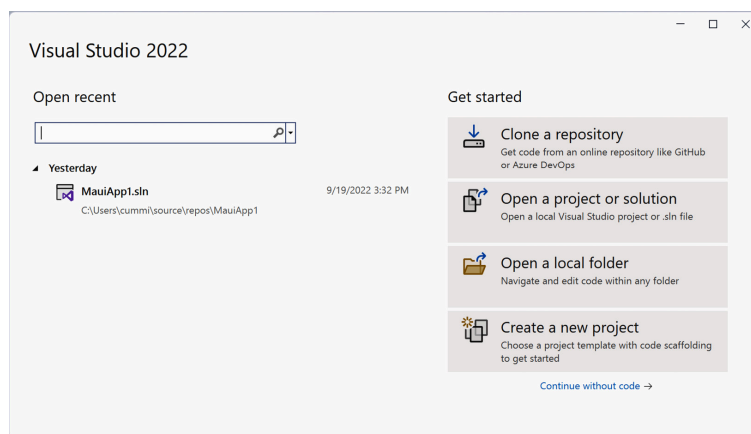


Figure 6.1 – Visual Studio 2022

This will open the **Create a new project** wizard.

2. In the search field, type in **maui** and select the **.NET MAUI App** item from the list:

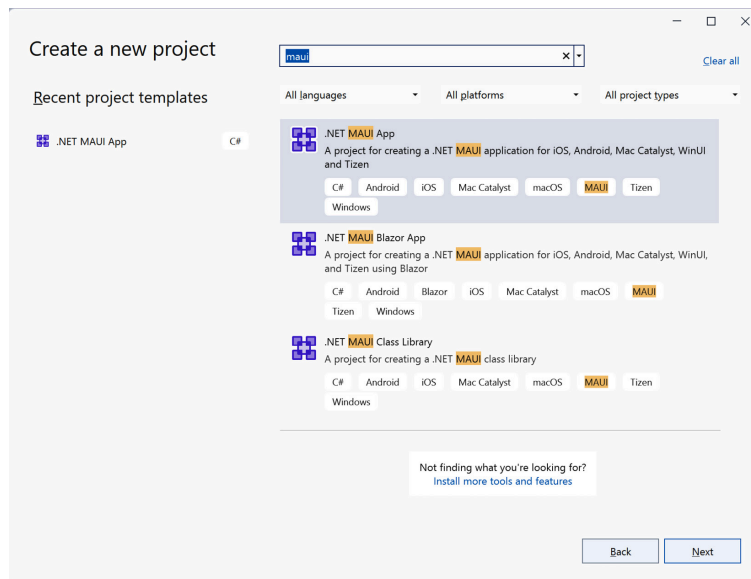


Figure 6.2 – Create a new project

3. Click **Next**.

4. Complete the next step of the wizard by naming your project. We will be calling our application **GalleryApp** in this case. Move on to the next dialog box by clicking **Next**, as shown in the following screenshot:

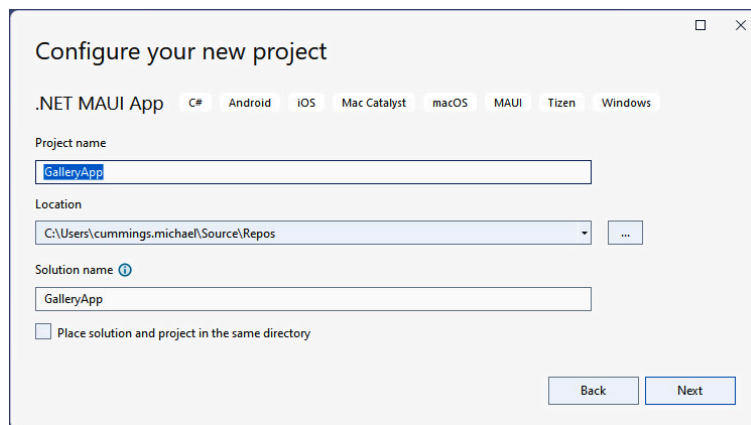


Figure 6.3 – Configure your new project

5. Click **Next**.

6. The last step will prompt you for the version of .NET Core to support. At the time of writing, .NET 6 is available as **Long-Term Support (LTS)**, and .NET 7 is available as **Standard-Term Support**. For the purposes of this book, we will assume that you will be using .NET 7.

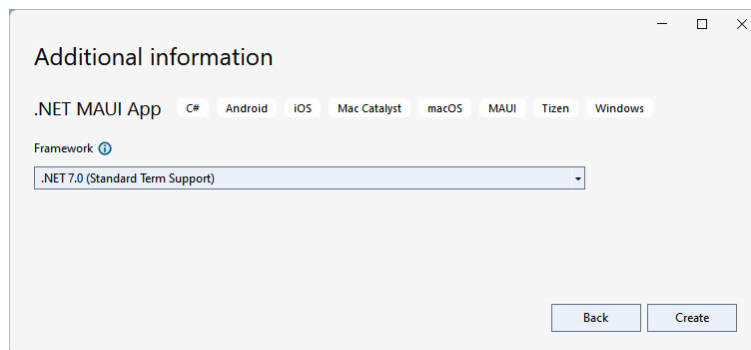


Figure 6.4 – Additional information

7. Finalize the setup by clicking **Create** and wait for Visual Studio to create the project.

Just like that, the app is created. Let's start by getting some photos to display.

Importing photos

The importing of photos is something that is carried out for all the platforms, so we will create a photo importer interface. The interface will have two **Get** methods—one that supports paging and one that gets photos with specified filenames. Both methods will also take a quality argument, but we will only use that argument in the iOS photo importer. The quality argument will be an **enum** type with two options—**High** and **Low**. However, before we create the interface, we will create a model class that will represent an imported photo using the following steps:

1. Create a new folder named **Models** in the **GalleryApp** project.
2. Create a new class named **Photo** in the recently created folder:

```
namespace GalleryApp.Models;
public class Photo
{
    public string Filename { get; set; }
    public byte[] Bytes { get; set; }
}
```

Now that we have created the model class, we can continue to create the interface:

1. Create a new folder named **Services** in the project.
2. Create a new interface named **IPhotoImporter** in the **Services** folder:

```
namespace GalleryApp.Services;
using System.Collections.ObjectModel;
using GalleryApp.Models;
public interface IPhotoImporter
{
    Task<ObservableCollection<Photo>> Get(int start, int count,
    Quality quality = Quality.Low);
    Task<ObservableCollection<Photo>> Get(List<string> filenames,
    Quality quality = Quality.Low);
}
```

3. In the **Services** folder, add a new file and create an **enum** type named **Quality** with two members—**Low** and **High**:

```
namespace GalleryApp.Services;
public enum Quality
{
    Low,
    High
}
```

4. In the **Services** folder, create a new class named **PhotoImporter**:

```
namespace GalleryApp.Services;
using GalleryApp.Models;
using System.Collections.ObjectModel;
internal partial class PhotoImporter : IPhotoImporter
{
    private partial Task<string[]> Import();
    public partial Task<ObservableCollection<Photo>> Get(int
start, int count, Quality quality);
    public partial Task<ObservableCollection<Photo>>
Get(List<string> filenames, Quality quality);
}
```

This class provides us with the base for the platform-specific implementations. By marking it **partial**, we are telling the compiler that there is more to this class in other files. We will be putting the implementation in the platform-specific folders later.

Now that we have the interface, we can add app permissions.

Requesting app permissions

If your app doesn't require any of the device's extra features, such as location, camera, or internet, then you will need to use permissions to request access to those resources. While each platform implements permissions slightly differently, .NET MAUI maps the platform-specific permissions into a common set of permissions to make things simpler. The permission system in .NET MAUI is also extensible so that you can create your own permissions to best suit your app.

Let's look at a specific example to see how requesting permissions works. **GalleryApp** displays images from a device's photo library. In the case of iOS and Android, the app must declare and request access to the photo library before it can use it. While these permissions are configured and named differently, .NET MAUI defines a **Photo** permission that hides those implementation details.

Follow these steps to add a permission check to **GalleryApp**:

1. Create a new class named **AppPermissions** in the **GalleryApp** project.
2. Modify the class definition to add a **partial** modifier, and remove the default constructor:

```
namespace GalleryApp;
internal partial class AppPermissions
{
}
```

3. Add the following class definition to the **AppPermissions** class:

```
internal partial class AppPermissions
{
    internal partial class AppPermission : Permissions.Photos
    {
    }
}
```

This creates a type named **AppPermission** that derives from the default .NET MAUI **Photos** permission class. It is also marked **partial** to allow for platform-specific implementation details to be added. Spoiler alert: we will need to have some platform-specific permissions.

4. Add the following method to the **AppPermissions** class:

```
public static async Task<PermissionStatus>
CheckRequiredPermission() => await Permissions.
CheckStatusAsync<AppPermission>();
```

The **CheckRequiredPermission** method is used to ensure our app has the right permissions before we attempt any operations that might fail if we don't. Its implementation is to call the .NET MAUI **CheckSyncStatus** with our **AppPermission** type. It returns a **PermissionStatus**, which is an **enum** type. We are mostly interested in the **Denied** and **Granted** values.

5. Add the **CheckAndRequestRequiredPermission** method to the **AppPermissions** class:

```
public static async Task<PermissionStatus>
CheckAndRequestRequiredPermission()
{
    PermissionStatus status = await Permissions.
CheckStatusAsync<AppPermission>();
    if (status == PermissionStatus.Granted)
        return status;
    if (status == PermissionStatus.Denied && DeviceInfo.Platform
== DevicePlatform.iOS)
    {
        // Prompt the user to turn on in settings
        // On iOS once a permission has been denied it may not be
        requested again from the application
        await App.Current.MainPage.DisplayAlert("Required App
Permissions", "Please enable all permissions in Settings for
this App, it is useless without them.", "Ok");
    }
    if
    (Permissions.ShouldShowRationale<AppPermission>())
    {
        // Prompt the user with additional information as to why the
        permission is needed
        await App.Current.MainPage.DisplayAlert("Required App
Permissions", "This is a Photo gallery app, without these
permissions it is useless.", "Ok");
    }
    status = await MainThread.InvokeOnMainThreadAsync(Permissions.
RequestAsync<AppPermission>);
    return status;
}
```

The **CheckAndRequestRequiredPermission** method handles the intricacies of requesting access from the user. The first step is to simply check whether the permission has already been granted, and if it has, return the status. Next, if you are on iOS and the permission has been denied, it cannot be requested again, so you must instruct the user on how to grant permission to the app by using the settings panel. In the

request behavior, Android includes the ability to nag the user if they have denied access. This behavior is exposed through .NET MAUI with the `ShouldShowRationale` method. It will return `false` for any platform that does not support this behavior; on Android, it will return `true` the first time the user denies access and `false` if the user denies it a second time. Finally, we request access for `AppPermission` from the user. Again, .NET MAUI is hiding all the platform implementation details from us, making checking and requesting access to certain resources very straightforward.

LOOK FAMILIAR?

If the preceding code looks familiar, it might be. This is the same implementation that is described in the .NET MAUI documentation. You can find it at <https://learn.microsoft.com/en-us/dotnet/maui/platform-integration/appmodel/permissions>.

Now that we have the shared `AppPermissions` in place, we can start with the platform implementations.

Importing photos from the iOS photo gallery

First, we will write the iOS code. To access photos, we need permission from the user, and we need to explain why we are asking for permission. To do that, we add text that explains why we need permission to the `info.plist` file. This text will be displayed when we ask the users for permission. To open the `info.plist` file, right-click on the file in the **Platforms/iOS** folder and click **Open With...**, then select **XML (Text) Editor**; this bypasses the default graphical `Info.plist` editor. Add the following text to the end of the `<dict>` element:

```
<key> NSPhotoLibraryUsageDescription </key>
<string> We want to show your photos in this app </string>
```

The first thing we will do is implement the `Import` method that reads what photos can be loaded:

1. In the **GalleryApp** project in the **Platforms/iOS** folder, create a new class called **PhotoImporter**.
2. Change the namespace declaration from `GalleryApp.Platforms.iOS` to `GalleryApp.Services`.
3. The partial class definitions must be in the same namespace, even though they are in different folders.
4. Add the **partial** modifier.
5. Resolve all the references.
6. Create a **private** field with a `PHAsset` dictionary named `assets`. This will be used to store photo information:

```
private Dictionary<string, PHAsset> assets;
```

7. Create a new **private partial** method named `Import`:

```
private partial async Task<string[]> Import()
{
}
```

8. In the **Import** method, request authorization using the **AppPermissions.CheckAndRequestRequiredPermission** method:

```
var status = await AppPermissions.
CheckAndRequestRequiredPermission();
```

9. If the user has granted access, fetch all the image assets by using **PHAsset.FetchAssets**:

```
internal partial class PhotoImporter
{
    private Dictionary<string, PHAsset> assets;
    private partial async Task<string[]> Import()
    {
        var status = await AppPermissions.
CheckAndRequestRequiredPermission();
        if (status == PermissionStatus.Granted)
        {
            assets = PHAsset.FetchAssets(PHAssetMediaType.Image, null)
                .Select(x => (PHAsset)x)
                .ToDictionary(asset => asset.
ValueForKey((NSString)"filename").ToString(), asset => asset);
        }
        return assets?.Keys.ToList().ToArray();
    }
}
```

Now, we have fetched **PHAssets** for all the photos, but to show the photo, we need to get the actual photo. On iOS, to do that, we need to request the image for the asset. This is something that is carried out asynchronously, so we will use **ObservableCollection**:

```
private void AddImage(ObservableCollection<Photo> photos, string path,
PHAsset asset, Quality quality)
{
    var options = new PHImageRequestOptions()
    {
        NetworkAccessAllowed = true,
        DeliveryMode = quality == Quality.Low ?
PHImageRequestOptionsDeliveryMode.FastFormat :
PHImageRequestOptionsDeliveryMode.HighQualityFormat
    };
    PHImageManager.DefaultManager.RequestImageForAsset(asset,
PHImageManager.MaximumSize, PHImageContentMode.AspectFill, options,
(image, info) =>
    {
        using NSData imageData = image.AsPNG();
        var bytes = new byte[imageData.Length];
        System.Runtime.InteropServices.Marshal.Copy(imageData.
Bytes, bytes, 0, Convert.ToInt32(imageData.Length));
        photos.Add(new Photo()
        {
            Bytes = bytes,
```



```

        Filename = Path.GetFileName(path)
    });
    });
}

```

Now, we have what we need to start implementing the two **Get** methods from the interface. We will start with the partial

Task<ObservableCollection<Photo>> Get(int start, int count, Quality quality = Quality.Low) method, which will be used to get photos from the **CollectionView** view that loads photos incrementally:

```

public partial async Task<ObservableCollection<Photo>> Get(int start,
int count, Quality quality)
{
    var photos = new ObservableCollection<Photo>();
    var status = await AppPermissions.
CheckAndRequestRequiredPermission();
    if (status == PermissionStatus.Granted)
    {
        var result = await Import();
        if (result.Length == 0)
        {
            return photos;
        }
        Index startIndex = start;
        Index endIndex = start + count;
        if (endIndex.Value >= result.Length)
        {
            endIndex = result.Length;
        }
        if (startIndex.Value > endIndex.Value)
        {
            return photos;
        }
        foreach (var path in result[startIndex..endIndex])
        {
            AddImage(photos, path, assets[path], quality);
        }
    }
    return photos;
}

```

The other method from the **IPhotoImporter** interface, **Task<ObservableCollection<Photo>> Get(List<string> filenames, Quality quality = Quality.Low)**, is very similar to the **Task<ObservableCollection<Photo>> Get(int start, int count, Quality quality = Quality.Low)** method. The only difference is that there is no code to handle indexes, and the **foreach** loop that loops through the result's array contains an **if** statement that checks whether the filename is the same as the current **PHAsset** object, and if it is, it calls the **AddImage** method:

```

public partial async Task<ObservableCollection<Photo>>
Get(List<string> filenames, Quality quality)
{
    var photos = new ObservableCollection<Photo>();

```

```

var result = await Import();
if (result?.Length == 0)
{
    return photos;
}
foreach (var path in result)
{
    if (filenames.Contains(path))
    {
        AddImage(photos, path, assets[path], quality);
    }
}
return photos;
}

```

In the preceding code, we set **NetworkAccessAllowed = true**. We do this to make it possible to download photos from **iCloud**.

Now, one of the four photo importers of our project is complete. The Mac Catalyst importer will be the next one we implement.

Importing photos from the Mac Catalyst photo gallery

The Mac Catalyst importer is exactly the same as what we just did for iOS. However, there isn't a convenient way to say, *"I need this class for just iOS and Mac Catalyst, and nothing else."* So we will take the path of least resistance and just copy the class into the Mac Catalyst platform folder:

1. Right-click the **PhotoImporter.cs** file in the **Platforms/iOS** folder in the project and select **Copy**.
2. Right-click the **Platforms/MacCatalyst** folder and select **Paste**.
3. Right-click on the **Info.plist** file in the **Platforms/MacCatalyst** folder and click **Open With...**, then select **XML (Text) Editor**, and add the following text to the end of the **<dict>** element:

```

<key> NSPhotoLibraryUsageDescription </key>
<string> We want to show your photos in this app </string>

```

That concludes the Mac Catalyst implementation of the **PhotoImporter** class. Next, we will work on the Android platform.

Importing photos from the Android photo gallery

Now that we have created an implementation for iOS, we will do the same for Android. Before we jump right into the importer, we need to address the permissions on Android.

In Android API version 33, three new permissions were added to enable read access to media files: **ReadMediaImages**, **ReadMediaVideos**, and **ReadMediaAudio**. Prior to API version 33, all that was required was the **ReadExternalStorage** permission. To properly request the correct permission for the API version of the device, create a new file named **AppPermissions** in the **Platform/Android** folder and modify it to look like the following:

```

using Android.OS;
[assembly: Android.App.UsesPermission(Android.Manifest.Permission.
ReadMediaImages)]
[assembly: Android.App.UsesPermission(Android.Manifest.Permission.
ReadExternalStorage, MaxSdkVersion = 32)]
namespace GalleryApp;
internal partial class AppPermissions
{
    internal partial class AppPermission : Permissions.Photos
    {
        public override (string androidPermission, bool isRuntime)[]
RequiredPermissions
        {
            get
            {
                List<(string androidPermission, bool isRuntime)> perms = new();
                if (Build.VERSION.SdkInt >= BuildVersionCodes.Tiramisu)
                    perms.Add((global::Android.Manifest.Permission.
ReadMediaImages, true));
                else
                    perms.Add((global::Android.Manifest.Permission.
ReadExternalStorage, true));
                return perms.ToArray();
            }
        }
    }
}

```

The first two lines add the required permissions to the **AndroidManifest.xml** file, similar to what we did manually with the iOS **info.plist** file. However, we only need **ReadMediaImages** for API 33+ and **ReadExternalStorage** for API versions below 33, so we set **MaxSdkVersion** for the **ReadExternalStorage** property. Then, we extend the **AppPermission** class with an implementation of the **RequirePermissions** property. In **RequirePermissions**, we return an array containing the **ReadMediaImages** permissions if the API version is 33 or higher, or the **ReadExternalStorage** permission if the API version is lower than 33. The Boolean value that is part of the **perms** array indicates whether the permission requires requesting access at runtime from the user. Now, when the app launches, it will request access for the correct permission based on the API level of the device.

Now that we have the Android-specific permissions sorted, we can import images using the following steps:

1. Create a new class named **PhotoImporter** in the project in the **Platforms/Android** folder.
2. Change the namespace declaration from **GalleryApp.Platforms.Android** to **GalleryApp.Services**.
3. The partial class definitions must be in the same namespace, even though they are in different folders.
4. Add the **partial** modifier.
5. Add a **using** statement for **GalleryApp.Models** to use the **Photo** class later.

6. Like the iOS implementation, we will start by implementing the

Import method. Add a new method named **Import** as follows:

```
private partial async Task<string[]> Import()
{
    var paths = new List<string>();
    return paths.ToArray();
}
```

7. Request permissions from the user to get the photos (highlighted in the following code block):

```
private partial async Task<string[]> Import()
{
    var paths = new List<string>();
    var status = await AppPermissions.
CheckAndRequestRequiredPermission();
    if (status == PermissionStatus.Granted)
    {
    }
    return paths.ToArray();
}
```

8. Now, use **ContentResolver** to query for the files and add them to the result:

```
private partial async Task<string[]> Import()
{
    var paths = new List<string>();
    var status = await AppPermissions.
CheckAndRequestRequiredPermission();
    if (status == PermissionStatus.Granted)
    {
        var imageUri = MediaStore.Images.Media.ExternalContentUri;
        var projection = new string[] { MediaStore.IMediaColumns.
Data };
        var orderBy = MediaStore.Images.IImageColumns.DateTaken;
        var cursor = Platform.CurrentActivity.ContentResolver.
Query(imageUri, projection, null, null, orderBy);
        while (cursor.MoveNext())
        {
            string path = cursor.GetString(cursor.
GetColumnIndex(MediaStore.IMediaColumns.Data));
            paths.Add(path);
        }
    }
    return paths.ToArray();
}
```

We will then start editing the **Task<ObservableCollection<Photo>>**

Get(int start, int count, Quality quality = Quality.Low) method.

If the import succeeds, we will continue to write the code that handles which photos should be imported in this loading of images. Conditions are specified with the **start** and **count** parameters. Use the following code listing to implement the first **Get** method:

```

public partial async Task<ObservableCollection<Photo>> Get(int start,
int count, Quality quality)
{
    var photos = new ObservableCollection<Photo>();
    var result = await Import();
    if (result.Length == 0)
    {
        return photos;
    }
    Index startIndex = start;
    Index endIndex = start + count;
    if (endIndex.Value >= result.Length)
    {
        endIndex = result.Length;
    }
    if (startIndex.Value > endIndex.Value)
    {
        return photos;
    }
    foreach (var path in result[startIndex..endIndex])
    {
        photos.Add(new()
        {
            Bytes = File.ReadAllBytes(path),
            Filename = Path.GetFileName(path)
        });
    }
    return photos;
}

```

Let's review the preceding code. The first step is to call the **Import** method and verify that there are photos to import. If there are none, we simply return an empty list. If there are photos to import, then we need to know **startIndex** and **endIndex** in the **photos** array to import. The code defaults to **endIndex** being **startIndex** plus the count of photos to import. If the count of photos to import is greater than the number of photos returned from the **Import** method, then **endindex** is adjusted to the length of the photos returned from the **Import** method. If **startIndex** is greater than **endIndex**, then we return the list of photos. Finally, we can read the images from **startIndex** to **endIndex** from the array of photos and return the bytes from the file and just the name of the file for each entry.

Now, we will continue with the other

Task<ObservableCollection<Photo>> Get (List<string> filenames, Quality quality = Quality.Low) method.

Create a **foreach** loop to loop through all the photos and to check whether each photo is specified in the **filenames** parameter. If a photo is specified in the **filenames** parameter, read the photo from the path, as in the first **Get** method:

```

public partial async Task<ObservableCollection<Photo>>
Get(List<string> filenames, Quality quality)
{
    var photos = new ObservableCollection<Photo>();

```

```

var result = await Import();
if (result.Length == 0)
{
    return photos;
}
foreach (var path in result)
{
    var filename = Path.GetFileName(path);
    if (!filenames.Contains(filename))
    {
        continue;
    }
    photos.Add(new Photo()
    {
        Bytes = File.ReadAllBytes(path),
        Filename = filename
    });
}
return photos;
}

```

With the Android importer finished, we can move to the final importer for Windows.

Importing photos from the Windows photo gallery

The final importer that we need is for the Windows platform. The code will follow the same pattern as for the other platforms; however, for Windows, we will use the **Windows Search** service to get the list of photos. Let's see how this platform is implemented by following these steps:

1. Import the **tlbimp-Windows.Search.Interop** and **System.Data.OleDb** NuGet packages. These are needed to search the filesystem for images.
2. Open the **GalleryApp** project by double-clicking it in **Solution Explorer**; edit the new imports to add a condition:

```

<PackageReference Include="System.Data.OleDb" Version="7.0.0"
  Condition="$([MSBuild]::GetTargetPlatformIdentifier('$(TargetFramework)')) == 'windows'" />
<PackageReference Include="tlbimp-Microsoft.Search.Interop"
  Version="1.0.0" Condition="$([MSBuild]::GetTargetPlatformIdentifier('$(TargetFramework)')) == 'windows'" />

```

We will use these two NuGet packages to search for image files on the local computer. But since we only need these assemblies on Windows, we limit **PackageReference** so that it is only used when **TargetPlatformIdentifier** is **'windows'**.

3. Create a new class called **PhotoImporter** in the **Windows** platform folder, and mark it as **partial**.
4. Change the namespace declaration from **GalleryApp.Platforms.Windows** to **GalleryApp.Services**.
5. The **partial** class definitions must be in the same namespace, even though they are in different folders.
6. Add **using** directives, so that we can use the classes in those namespaces:

```
using GalleryApp.Models;
```

```
using Microsoft.Search.Interop;
using System.Data.OleDb;
```

7. Add a **private** field to hold the **QueryHelper** reference:

```
ISearchQueryHelper queryHelper;
```

8. Like the previous implementations, we will start by implementing the **Import** method, so add a new method named **Import** as follows:

```
private partial async Task<string[]> Import()
{
    var paths = new List<string>();
    return paths.ToArray();
}
```

9. Request permissions from the user to get the photos (highlighted in the following code block):

```
private partial async Task<string[]> Import()
{
    var paths = new List<string>();
    var status = await AppPermissions.
CheckAndRequestRequiredPermission();
    if (status == PermissionStatus.Granted)
    {
    }
    return paths.ToArray();
}
```

10. Now, using **QueryHelper**, get all the image paths:

```
private partial async Task<string[]> Import()
{
    var paths = new List<string>();
    var status = await AppPermissions.
CheckAndRequestRequiredPermission();
    if (status == PermissionStatus.Granted)
    {
        string sqlQuery = queryHelper.GenerateSQLFromUserQuery(" ");
        using OleDbConnection conn = new(queryHelper.
ConnectionString);
        conn.Open();
        using OleDbCommand command = new(sqlQuery, conn);
        using OleDbDataReader WDSResults = command.ExecuteReader();
        while (WDSResults.Read())
        {
            var itemUrl = WDSResults.GetString(0);
            paths.Add(itemUrl);
        }
    }
    return paths.ToArray();
}
```

Here, **QueryHelper** is used to create a SQL query, and we use **OleDbConnection** to query the search index for all the matching files.

We can now start editing the **Task<ObservableCollection<Photo>>**
Get(int start, int count, Quality quality = Quality.Low) method.

Add the following declaration to the **PhotoImporter** class:

```
public partial async Task<ObservableCollection<Photo>> Get(int start,
int count, Quality quality)
{
}
```

Now, we will start the implementation of the method by setting up file patterns and the locations we will search:

```
string[] patterns = { ".png", ".jpeg", ".jpg" };
string[] locations = {
    Environment.GetFolderPath(Environment.SpecialFolder.MyPictures),
    Environment.GetFolderPath(Environment.SpecialFolder.
        CommonPictures),
    Path.Combine(Environment.GetFolderPath(Environment.SpecialFolder.
        UserProfile), "OneDrive", "Camera Roll")
};
```

These arrays define the file extensions and folders that we will search for photos. We will then create **QueryHelper** from the **tlbimp-Windows.Search.Interop** NuGet package and, using the arrays, configure the query parameters:

```
queryHelper = new CSearchManager().GetCatalog("SystemIndex").
    GetQueryHelper();
queryHelper.QueryMaxResults = start + count;
queryHelper.QuerySelectColumns = "System.ItemPathDisplay";
queryHelper.QueryWhereRestrictions = "AND (";
foreach (var pattern in patterns)
    queryHelper.QueryWhereRestrictions += " Contains(System.
        FileExtension, '" + pattern + "') OR";
queryHelper.QueryWhereRestrictions = queryHelper.
    QueryWhereRestrictions[..^2];
queryHelper.QueryWhereRestrictions += ")";
queryHelper.QueryWhereRestrictions += " AND (";
foreach (var location in locations)
    queryHelper.QueryWhereRestrictions += " scope='" + location + "'
OR";
queryHelper.QueryWhereRestrictions = queryHelper.
    QueryWhereRestrictions[..^2];
queryHelper.QueryWhereRestrictions += ")";
queryHelper.QuerySorting = "System.DateModified DESC";
```

QueryMaxResults is set so that we only retrieve the results we are looking for. Then, we specify that the only data column to return is **"System.ItemPathDisplay"**. Next, we set **QueryWhereRestrictions** from our list of extensions. Note the use of the **range** operator to remove the trailing **"OR"** in the query string. We use the same technique to add the locations to **QueryWhereRestrictions**. Finally, we set the sort order.

The remainder of the method is going to be very similar to those of the previous platforms. If the import succeeds, we will continue to handle what photos should be imported in this loading of images. Conditions are

specified with the **start** and **count** parameters. Use the following code listing to complete the implementation of the first **Get** method:

```
var photos = new ObservableCollection<Photo>();
var result = await Import();
if (result?.Length == 0)
{
    return photos;
}
Index startIndex = start;
Index endIndex = start + count;
if (endIndex.Value >= result.Length)
{
    endIndex = result.Length;
}
if (startIndex.Value > endIndex.Value)
{
    return photos;
}
foreach (var uri in result[startIndex..endIndex])
{
    var path = new System.Uri(uri).AbsolutePath;
    photos.Add(new()
    {
        Bytes = File.ReadAllBytes(path),
        Filename = Path.GetFileName(path)
    });
}
return photos;
```

Let's quickly review the preceding code. The first step is to call the **Import** method and verify that there are photos to import. If there are none, we simply return an empty list. If there are photos to import, then we need to know **startIndex** and **endIndex** in the **photos** array to import. **startIndex** and **endIndex** are adjusted to make sure they are valid for the photos to import. Then, we can read the images from **startIndex** to **endIndex** from the array of photos and return the bytes from the file and just the name of the file for each entry.

Now, we will continue with the other

Task<ObservableCollection<Photo>> Get(List<string> filenames, Quality quality = Quality.Low) method. Add the following declaration to the **PhotoImporter** class:

```
public partial async Task<ObservableCollection<Photo>>
Get(List<string> filenames, Quality quality)
{
}
```

Now, we will start the implementation of the method by setting the search parameters:

```
queryHelper = new CSearchManager().GetCatalog("SystemIndex").
GetQueryHelper();
```

```

queryHelper.QuerySelectColumns = "System.ItemPathDisplay";
queryHelper.QueryWhereRestrictions = "AND (";
foreach (var filename in filenames)
    queryHelper.QueryWhereRestrictions += " Contains(System.FileName,
'" + filename + "') OR";
queryHelper.QueryWhereRestrictions = queryHelper.
QueryWhereRestrictions[..^2];
queryHelper.QueryWhereRestrictions += ")";

```

For this method, we only need to add all the filenames to **QueryWhereRestrictions**. Following that, call the **Import** method, and if it returns results, then use a **foreach** loop to loop through all the photos and to check whether each photo is specified in the **filenames** parameter. If a photo is specified in the **filenames** parameter, read the photo from the path, as in the first **Get** method:

```

var photos = new ObservableCollection<Photo>();
var result = await Import();
if (result?.Length == 0)
{
    return photos;
}
foreach (var uri in result)
{
    var path = new System.Uri(uri).AbsolutePath;
    var filename = Path.GetFileName(path);
    if (filenames.Contains(filename))
    {
        photos.Add(new()
        {
            Bytes = File.ReadAllBytes(path),
            Filename = filename
        });
    }
}
return photos;

```

The photo importers are now finished, and we are ready to write the rest of the app, which will mostly involve adding code that is shared between the platforms.

Writing the app-initializing code

We have now written the code that we will use to get data to the app. Let's continue to build the app, starting with initializing the core parts of the app.

Wiring up dependency injection

By using dependency injection as a pattern, we can keep our code cleaner and more testable. This app will use constructor injection, which means that all the dependencies that a class has must be passed through its constructor. The container then constructs objects for you, so you don't have to care too much about the dependency chain. Since .NET MAUI already includes a dependency injection framework,

Microsoft.Extensions.DependencyInjection, there is nothing extra to install.

CONFUSED ABOUT DEPENDENCY INJECTION?

Check out the Wiring up a dependency injection section in Chapter 2, Building Our First .NET MAUI App, for more details on dependency injection.

While it is recommended to use extension methods to group the types together, we have very few types in this app to register so we will use a different method in the next section.

Registering PhotoImporter with dependency injection

Let's add the required code to register the types we have created so far, as shown in the following steps:

1. In the **GalleryApp** project, open **MauiProgram.cs**.
2. Make the following changes to the **MauiProgram** class (the changes are highlighted):

```
using GalleryApp.Services;
using Microsoft.Extensions.Logging;
public static class MauiProgram
{
    public static MauiApp CreateMauiApp()
    {
        var builder = MauiApp.CreateBuilder();
        builder
            .UseMauiApp<App>()
            .ConfigureFonts(fonts =>
            {
                fonts.AddFont("OpenSans-Regular.ttf",
                    "OpenSansRegular");
                fonts.AddFont("OpenSans-Semibold.ttf",
                    "OpenSansSemibold");
            });
        #if DEBUG
            builder.Logging.AddDebug();
        #endif
        builder.Services.AddSingleton<IPhotoImporter>(serviceProvider
            => new PhotoImporter());
        return builder.Build();
    }
}
```

The .NET MAUI **MauiAppBuilder** class exposes the **Services** property, which is the dependency injection container. We simply need to add the types we want dependency injection to know about and the container will do the rest for us. Think of a builder as something that collects a lot of information on what needs to be done, and then builds the object we need. It's a very useful pattern on its own, by the way.

We only use the builder for one thing at the moment. Later on, we will use it to register any class in the assembly that inherits from our abstract **ViewModel** class and our views. The container is now prepared for us to ask for these types.

Creating a shell

The main navigation for this app will be tabs at the bottom of the screen. The app will have a fly-out menu with two options—**Home** and **Gallery**:

1. Create a new folder named **Views** in the project.
2. In the **Views** folder, create two new files using the **.NET MAUI ContentPage (XAML)** template—one named **MainView** and one named **GalleryView**.
3. Delete the **MainPage.Xaml** and **MainPage.Xaml.cs** files from the root of the project since we won't be needing those.
4. Open the **AppShell.xaml** file in the root of the project.

Add both views to the **Shell** object using the **ContentTemplate** property of **ShellContent**. Use the **DataTemplate** markup extension to load the view from the dependency injection container:

```
<?xml version="1.0" encoding="UTF-8" ?>
<Shell
  x:Class="GalleryApp.AppShell"
  xmlns="http://schemas.microsoft.com/dotnet/2021/maui"
  xmlns:x="http://schemas.microsoft.com/winfx/2009/xaml"
  xmlns:local="clr-namespace:GalleryApp"
  xmlns:views="clr-namespace:GalleryApp.Views">
  <ShellContent Title="Home" ContentTemplate="{DataTemplate views:MainView}" />
  <ShellContent Title="Gallery" ContentTemplate="{DataTemplate views:GalleryView}" />
</Shell>
```

5. Since the views are loaded via **DataTemplates**, they must be registered with dependency injection. Add the highlighted code to **MauiProgram.cs**, after the **IPhotoImporter** line:

```
builder.Services.
AddSingleton<IPhotoImporter>(serviceProvider => new
PhotoImporter());
builder.Services.AddTransient<Views.MainView>();
builder.Services.AddTransient<Views.GalleryView>();
return builder.Build();
```

Now that we have created a shell, let's continue with some other base code before we start to create the views.

Creating a base view model

Before we create an actual view model, we will create an abstract base view model that all view models can inherit from. The idea behind this base view model is that we can write common code in it. In this case, we will implement the **INotifyPropertyChanged** interface by going through the following steps:

1. In the **GalleryApp** project, create a folder named **ViewModels**.
2. Add a NuGet reference to **CommunityToolkit.Mvvm**; we use **CommunityToolkit.Mvvm** for implementing the **INotifyPropertyChanged** interface, as we have in other chapters.
3. Create a new abstract class named **ViewModel**:

```

namespace GalleryApp.ViewModels;
using CommunityToolkit.Mvvm.ComponentModel;
public abstract partial class ViewModel: ObservableObject
{
    [ObservableProperty]
    [NotifyPropertyChangedFor(nameof(IsNotBusy))]
    private bool isBusy;
    public bool IsNotBusy => !IsBusy;
    abstract protected internal Task Initialize();
}

```

In this app's **ViewModel** class, we have added an abstract method for **Initialize**. Each **ViewModel** implementation will override this method and load the images asynchronously for display. The **IsBusy** and **NotIsBusy** properties are used as flags to indicate when the data has completed loading.

Now, we have a **ViewModel** base that we can use for all **ViewModel** instances that we will create later in this project.

Creating the gallery view

Now, we will start to build the views. We will start with the gallery view, which will show the photos as a grid. We will start with **GalleryViewModel**, and then create **GalleryView**. Creating the view model first allows Visual Studio to use the **GalleryViewModel** definition to check the syntax of the data bindings in the XAML file.

Creating GalleryViewModel

GalleryViewModel is the class that will be responsible for fetching the data and handling the view logic. Because photos will be added asynchronously to the photo collection, we don't want to set **IsBusy** to **false** immediately after we call the **Get** method of **PhotoImporter**. We will instead wait 3 seconds first. However, we will also add an event listener to the collection so that we can listen for changes. If the collection changes and there are items in it, we will set **IsBusy** to **false**. Create a class named **GalleryViewModel** in the **ViewModels** folder and add the following code to implement this:

```

namespace GalleryApp.ViewModels;
using CommunityToolkit.Mvvm.ComponentModel;
using GalleryApp.Models;
using GalleryApp.Services;
using System.Collections.ObjectModel;
using System.Threading.Tasks;
public partial class GalleryViewModel : ViewModel
{
    private readonly IPhotoImporter photoImporter;
    [ObservableProperty]
    public ObservableCollection<Photo> photos;
    public GalleryViewModel(IPhotoImporter photoImporter) : base()
    {

```

```

        this.photoImporter = photoImporter;
    }
    override protected internal async Task Initialize()
    {
        IsBusy = true;
        Photos = await photoImporter.Get(0, 20);
        Photos.CollectionChanged += Photos_CollectionChanged;
        await Task.Delay(3000);
        IsBusy = false;
    }
    private void Photos_CollectionChanged(object sender, System.
Collections.Specialized.NotifyCollectionChangedEventArgs e)
    {
        if (e.NewItems != null && e.NewItems.Count > 0)
        {
            IsBusy = false;
            Photos.CollectionChanged -= Photos_CollectionChanged;
        }
    }
}

```

Finally, register **GalleryViewModel** with dependency injection in **MauiProgram**:

```

        builder.Services.AddSingleton<IphotoImporter>(serviceProvider
=> new PhotoImporter());
        builder.Services.AddTransient<ViewModels.GalleryViewModel>();
        builder.Services.AddTransient<Views.MainView>();
        builder.Services.AddTransient<Views.GalleryView>();
        return builder.Build();

```

Now, **GalleryViewModel** is ready, so we can start to create **GalleryView**.

Creating GalleryView

First, we will create a converter that will convert **byte[]** to **Microsoft.Maui.Controls.ImageSource**. In the **GalleryApp** project, create a new folder named **Converters**, and inside the folder, create a new class named **BytesToImageConverter**:

```

namespace GalleryApp.Converters;
using System.Globalization;
internal class BytesToImageConverter : IValueConverter
{
    public object Convert(object value, Type targetType, object
parameter, CultureInfo culture)
    {
        if (value != null)
        {
            var bytes = (byte[])value;
            var stream = new MemoryStream(bytes);
            return ImageSource.FromStream(() => stream);
        }
        return null;
    }
    public object ConvertBack(object value, Type targetType, object
parameter, CultureInfo culture)

```

```

    {
        throw new NotImplementedException();
    }
}

```

To use the converter, we need to add it as a resource. We will do this by adding it to a **Resource Dictionary** object in the **Resources** property of **GalleryView**.

Open **GalleryView.xaml**, and add the following highlighted code to the view:

```

<ContentPage xmlns="http://schemas.microsoft.com/dotnet/2021/maui"
              xmlns:x="http://schemas.microsoft.com/winfx/2009/xaml"
              xmlns:converters="clr-namespace:GalleryApp.Converters"
              x:Class="GalleryApp.Views.GalleryView"
              Title="GalleryView">
  <ContentPage.Resources>
    <ResourceDictionary>
      <converters:BytesToImageConverter x:Key="ToImage" />
    </ResourceDictionary>
  </ContentPage.Resources>
</ContentPage>

```

To be able to bind to **ViewModel**, we will set **BindingContext** to **GalleryViewModel**. Use the constructor dependency injection in **GalleryView.xaml.cs** to create an instance of **GalleryViewModel**.

Open **GalleryView.xaml.cs**, and add the following highlighted code to the class:

```

public GalleryView(GalleryViewModel viewModel)
{
    InitializeComponent();
    BindingContext = viewModel;
    MainThread.InvokeOnMainThreadAsync(viewModel.Initialize);
}

```

First, we modify the constructor so that dependency injection will provide an instance of **GalleryViewModel**. That instance is set as **BindingContext** for the page. This object will be used in the XAML bindings of the view. Finally, we initialize the view model asynchronously.

What we will show in this view is a grid with three columns. To build this with .NET MAUI, we will use the **CollectionView** control. To specify the layout that **CollectionView** should have, add a **GridItemsLayout** element to the **ItemsLayout** property of **CollectionView**. Follow these steps to build this view:

1. Navigate to **GalleryView.xaml**.

Import the namespaces for **GalleryApp.ViewModels** and **GalleryApp.Models** as **viewModels** and **models**, respectively:

```

xmlns:converters="clr-namespace:GalleryApp.Converters"
xmlns:models="clr-namespace: GalleryApp.Models"

```

```
xmlns:viewModels="clr-namespace: GalleryApp.ViewModels"
x:Class=" GalleryApp.Views.GalleryView"
```

2. On **ContentPage**, set **x:DataType** to **viewModels:GalleryViewModel**.

This makes the bindings compile, which will make our view faster to render:

```
<CollectionView x:Name="Photos" ItemsSource="{Binding Photos}">
  <CollectionView.ItemsLayout>
    <GridItemsLayout Orientation="Vertical" Span="3"
HorizontalItemSpacing="0" />
  </CollectionView.ItemsLayout>
  <CollectionView.ItemTemplate>
    <DataTemplate x:DataType="models:Photo">
      <Grid>
        <Image Aspect="AspectFill" Source="{Binding Bytes,
Converter={StaticResource ToImage}}" HeightRequest="120" />
      </Grid>
    </DataTemplate>
  </CollectionView.ItemTemplate>
</CollectionView>
```

Now, we can see the photos in the view. However, we will also need to create the content that will be shown when we don't have any photos to show as they have not been loaded yet, or if there are no photos available. Add the following highlighted code to create a **DataTemplate** object to show when **CollectionView** doesn't have any data:

```
<CollectionView :Name="Photos" ItemsSource="{Binding Photos}"
EmptyView="{Binding}">
...
  <CollectionView.EmptyViewTemplate>
    <DataTemplate x:DataType="viewModels:GalleryViewModel">
      <Grid>
        <ActivityIndicator IsVisible="{Binding IsBusy}" />
        <Label Text="No photos to import could be found"
IsVisible="{Binding IsNotBusy}" HorizontalOptions="Center"
VerticalOptions="Center" HorizontalTextAlignment="Center" />
      </Grid>
    </DataTemplate>
  </CollectionView.EmptyViewTemplate>
</CollectionView>
```

Now, we can run the app. The next step is to load more photos when a user reaches the end of the view.

Loading photos incrementally

To load more than the first 20 items, we will load photos incrementally so that when users scroll to the end of **CollectionView**, it will start to load more items. **CollectionView** has built-in support for loading data incrementally. Because we get an **ObservableCollection** object back from the photo importer and data is added asynchronously to it, we need to create an event listener to handle when items are added to the photo importer so that we can add it to the **ObservableCollection** instance that we bound to **CollectionView**. Create the event listener by navigating to

GalleryViewModel.cs and adding the following code at the end of the class:

```
private int itemsAdded;
private void Collection_CollectionChanged(object sender, System.
Collections.Specialized.NotifyCollectionChangedEventArgs args)
{
    foreach (Photo photo in args.NewItems)
    {
        itemsAdded++;
        Photos.Add(photo);
    }
    if (itemsAdded == 20)
    {
        var collection = (ObservableCollection<Photo>)sender;
        collection.CollectionChanged -= Collection_CollectionChanged;
    }
}
private int currentStartIndex = 0;
[RelayCommand]
public async Task LoadMore()
{
    currentStartIndex += 20;
    itemsAdded = 0;
    var collection = await photoImporter.Get(currentStartIndex, 20);
    collection.CollectionChanged += Collection_CollectionChanged;
}
```

The only thing we have left to do to get the incremental load to work is to bind **CollectionView** to the code we created in **ViewModel1**. The following code will trigger the loading of more photos when the user has just five items left:

```
<CollectionView x:Name="Photos" EmptyView="{Binding}"
ItemsSource="{Binding Photos}" RemainingItemsThreshold="5"
RemainingItemsThresholdReachedCommand="{Binding LoadMoreCommand}">
```

Now that we have a view that shows photos and loads them incrementally, we can make it possible to add photos as favorites.

Saving favorites

In **GalleryView**, we want to be able to select favorites that we can show in **MainView**. To do that, we need to store the photos that we have selected so that it remembers our selection. Create a new interface in the **GalleryApp** project named **ILocalStorage** in the **Services** folder:

```
public interface ILocalStorage
{
    void Store(string filename);
    List<string> Get();
}
```

The easiest way to store/persist data in .NET MAUI is to use the built-in property store. **Preferences** is a static class in the

Microsoft.Maui.Storage namespace. Follow these steps to use it:

1. Create a new class named **MauiLocalStorage** in the **Services** folder.
2. Implement the **ILocalStorage** interface:

```
namespace GalleryApp.Services;
using System.Text.Json;
public class MauiLocalStorage : ILocalStorage
{
    public const string FavoritePhotosKey = "FavoritePhotos";
    public List<string> Get()
    {
        if (Preferences.ContainsKey(FavoritePhotosKey))
        {
            var filenames = Preferences.Get(FavoritePhotosKey, string.Empty);
            return JsonSerializer.Deserialize<List<string>>(filenames);
        }
        return new List<string>();
    }
    public void Store(string filename)
    {
        var filenames = Get();
        filenames.Add(filename);
        var json = JsonSerializer.Serialize(filenames);
        Preferences.Set(FavoritePhotosKey, json);
    }
}
```

To be able to use **ILocalStorage** with constructor injection, we need to register it with the container. Navigate to the **MauiProgram** class and add the following highlighted code:

```
builder.Services.AddSingleton<IPhotoImporter>(serviceProvider => new
PhotoImporter());
builder.Services.AddTransient<ILocalStorage>(ServiceProvider => new
MauiLocalStorage());
builder.Services.AddTransient<ViewModels.MainViewModel>();
```

Now, we are ready to use the local storage.

Navigate to the **GalleryViewModel** class, add the **ILocalStorage** interface to the constructor, and assign it to a field:

```
private readonly IPhotoImporter photoImporter;
private readonly ILocalStorage localStorage;
public GalleryViewModel(IPhotoImporter photoImporter, ILocalStorage
localStorage)
{
    this.photoImporter = photoImporter;
    this.localStorage = localStorage;
}
```

The next step is to create a command that we can bind to from the view when we select photos. The command will monitor which photos we have selected and notify other views that we have added favorite photos. We

will use **WeakReferenceManager** from **CommunityToolkit** to send messages from **GalleryViewModel** to **MainViewModel**.

Follow these steps to implement the **GalleryViewModel** side:

1. Create a new class in the **Services** folder named **Messages**:

```
namespace GalleryApp.Services;
internal static class Messages
{
    public const string FavoritesAddedMessage =
        nameof(FavoritesAddedMessage);
}
```

This is used to define the message type we are sending to **MainViewModel**.

2. Navigate to **GalleryViewModel**.
3. Create a new method named **AddFavorites** that is attributed to the **RelayCommand** type.
4. Add the following code:

```
[RelayCommand]
public void AddFavorites(List<Photo> photos)
{
    foreach (var photo in photos)
    {
        localStorage.Store(photo.Filename);

        WeakReferenceMessenger.Default.Send<string>(Messages.
            FavoritesAddedMessage);
    }
}
```

Now, we are ready to start working with the view. The first thing we will do is make it possible to select photos. Navigate to **GalleryView.xaml** and set the **SelectionMode** mode of **CollectionView** to **Multiple** to make it possible to select multiple items:

```
<CollectionView x:Name="Photos"
    EmptyView="{Binding}" ItemsSource="{Binding Photos}"
    SelectionMode="Multiple" RemainingItemsThreshold="5"
    RemainingItemsThresholdReachedCommand="{Binding LoadMore}">
```

When a user selects a photo, we want it to be clear which photos have been selected. To achieve this, we will use **VisualStateManager**. We will do this by creating a style for **Grid** and setting **Opacity** to **0.5**, as in the following code. Add the code to **Resources** of the page:

```
<ContentPage.Resources>
    <ResourceDictionary>
        <converters:BytesToImageConverter x:Key="ToImage" />
        <Style TargetType="Grid">
            <Setter Property="VisualStateManager.VisualStateGroups">
                <VisualStateGroupList>
                    <VisualStateGroup x:Name="CommonStates">
```

```

        <VisualState x:Name="Normal" />
        <VisualState x:Name="Selected">
            <VisualState.Setters>
                <Setter Property="Opacity" Value="0.5" />
            </VisualState.Setters>
        </VisualState>
    </VisualStateGroup>
</VisualStateManager>
</Setter>
</Style>
</ResourceDictionary>
</ContentPage.Resources>

```

To save the selected photos, we will create a toolbar item that the user can tap:

1. Add **ToolbarItem** with the **Text** property set to **Select**.
2. Add an event handler named **SelectToolBarItem_Clicked**:

```

<ContentPage.ToolbarItems>
    <ToolbarItem Text="Select" Clicked="SelectToolBarItem_Clicked" />
</ContentPage.ToolbarItems>

```

3. Navigate to the code behind the **GalleryView.xaml.cs** file.
4. Add the following **using** statements:

```

using GalleryApp.Models;
using GalleryApp.ViewModels;

```

5. Create an event handler named **SelectToolBarItem_Clicked**:

```

private void SelectToolBarItem_Clicked(object sender, EventArgs e)
{
    if (!Photos.SelectedItems.Any())
    {
        DisplayAlert("No photos", "No photos selected", "OK");
        return;
    }
    var viewModel = (GalleryViewModel)BindingContext;
    viewModel.AddFavoritesCommand.Execute(Photos.SelectedItems.Select(x => (Photo)x).ToList());
    DisplayAlert("Added", "Selected photos have been added to favorites", "OK");
}

```

Now that we are done with **GalleryView**, we will continue with the main view, which will show the latest photos and the favorite photos in two carousels.

Creating the carousels for MainView

The last view in this app is **MainView**, which is the view that is visible when users start the app. This view will show two carousel views—one with recent photos and one with favorite photos.

Creating the view model for MainView

We will start by creating **ViewModel** that we will use for the view. In the **ViewModel** folder, create a new class named **MainViewModel**:

```
namespace GalleryApp.ViewModels;
using CommunityToolkit.Mvvm.ComponentModel;
using CommunityToolkit.Mvvm.Messaging;
using GalleryApp.Models;
using GalleryApp.Services;
using System.Collections.ObjectModel;
public partial class MainViewModel : ViewModel
{
    private readonly IPhotoImporter photoImporter;
    private readonly ILocalStorage localStorage;
    [ObservableProperty]
    private ObservableCollection<Photo> recent;
    [ObservableProperty]
    private ObservableCollection<Photo> favorites;
    public MainViewModel(IPhotoImporter photoImporter, ILocalStorage
localStorage)
    {
        this.photoImporter = photoImporter;
        this.localStorage = localStorage;
    }
    override protected internal async Task Initialize()
    {
        var photos = await photoImporter.Get(0, 20, Quality.Low);
        Recent = photos;
        await LoadFavorites();
        WeakReferenceMessenger.Default.Register<string>(this, async
(sender, message) => {
            if( message == Messages.FavoritesAddedMessage )
            {
                await MainThread.InvokeOnMainThreadAsync(LoadFavorites);
            }
        });
    }
    private async Task LoadFavorites()
    {
        var filenames = localStorage.Get();
        var favorites = await photoImporter.Get(filenames, Quality.Low);
        Favorites = favorites;
    }
}
```

In the preceding code, the **Initialize** method is used to register a call-back with **Weak ReferenceManager**. This callback invokes the **LoadFavorites** method if the message sent was **Message.FavoritesAddedMessage**. Recall that **Messages.FavoritesAddedMessage** is sent from **GalleryViewModel** after selecting new photos.

In the **LoadFavorites** method, the favorites are loaded from the storage provider instance in **localStorage**. Then, the photos from the favorites are imported using the **photoImporter** instance.

We need to add the view model to dependency injection so that we can use it in the view. Open **MauiProgram** and add the highlighted code:

```
builder.Services.AddTransient<ILocalStorage>(ServiceProvider
=> new MauiLocalStorage());
builder.Services.AddTransient<ViewModels.MainViewModel>();
builder.Services.AddTransient<ViewModels.GalleryViewModel>();
```

Now that we have created **MainViewModel**, we will continue with the latest photos.

Showing the latest photos

We are now ready to set up the carousel views. We have already created the view model, so we can use the view model to populate the view with content.

Let's look at the steps to create the view:

1. In the constructor of the code, behind the **MainView.xaml.cs** file, set **ViewModel** to **BindingContext**:

```
public MainView(MainViewModel viewModel)
{
    InitializeComponent();
    BindingContext = viewModel;
    MainThread.InvokeOnMainThreadAsync(viewModel.Initialize);
}
```

2. Navigate to **MainView.xaml**.
3. Add the following code:

```
<ContentPage xmlns="http://schemas.microsoft.com/dotnet/2021/maui"
             xmlns:x="http://schemas.microsoft.com/winfx/2009/xaml"
             xmlns:converters="clr-namespace:GalleryApp.Converters"
             xmlns:viewModels="clr-namespace:GalleryApp.ViewModels"
             xmlns:models="clr-namespace:GalleryApp.Models"
             x:Class="GalleryApp.Views.MainView"
             x:DataType="viewModels:MainViewModel"
             Title="My Photos">
    <ContentPage.Resources>
        <ResourceDictionary>
            <converters:BytesToImageConverter x:Key="ToImage" />
        </ResourceDictionary>
    </ContentPage.Resources>
    <Grid>
        <Grid.RowDefinitions>
            <RowDefinition Height="*" />
            <RowDefinition Height="50" />
            <RowDefinition Height="*" />
            <RowDefinition Height="20" />
        </Grid.RowDefinitions>
        <CarouselView ItemsSource="{Binding Recent}"
            PeekAreaInsets="40,0,40,0" >
            <CarouselView.ItemsLayout>
                <LinearItemsLayout Orientation="Horizontal" SnapPointsAlignment="Start"
                    SnapPointsType="Mandatory" />
            </CarouselView.ItemsLayout>
            <CarouselView.ItemTemplate>
                <DataTemplate x:DataType="models:Photo">
```

```

        <Image Source="{Binding Bytes,
        Converter={StaticResource ToImage}}" Aspect="AspectFill" />
    </DataTemplate>
</CarouselView.ItemTemplate>
</CarouselView>
</Grid>
</ContentPage>

```

The **CarouselView** control is used to present data to the user in a scrollable layout, where the user can swipe to move through the collection of items. It is very similar to **CollectionView**; however, the uses of the two controls are different. You would use **CollectionView** when you want to display a list of items with an indeterminate length, and **CarouselView** is used to highlight items from a list of items with a limited length. Since **CarouselView** shares implementations with the **CollectionView** control, it uses the familiar **ItemTemplate** property to customize how each item is displayed. It adds an **ItemsLayout** property to define how the collection of items is displayed. **CarouselView** can use either a **Horizontal** or **Vertical** layout direction, with **Horizontal** being the default.

In **MainView**, **CarouselView** is used to display the **Recent** photos from **MainViewModel**. **ItemsLayout** is customized to set the scrolling behavior so that items will snap into view using the start, or left edge of the image. The **SnapPointType** property set to **Mandatory** makes sure that **CarouselView** snaps the image into place after scrolling, which would ensure a single image is always in view.

ItemTemplate is used to display an image that is data-bound to each photo and displays the image from the bytes in the **Photo** model.

BytesToImageConverter converts the byte array from the **Photo** model into **ImageSource** that can be displayed by the **Image** control. The **Image** control has the **Aspect** property set to **AspectFill**, allowing the image control to resize the image, maintaining the aspect ratio of the source image to fill the available visible space.

Now that we have shown the latest photos in a carousel, the next (and the last) step is to show the favorite photos in another carousel.

Showing the favorite photos

The last thing we will do in this app is add a carousel to show favorite photos. Add the following highlighted code inside **Grid**, after the first **CarouselView**, as shown in the following code snippet:

```

<Grid>
    <!--Code omitted for brevity -->
    <CarouselView>
        <!--Code omitted for brevity -->
    </CarouselView>
    <Label Grid.Row="1" Margin="10" Text="Favorites" FontSize="Subtitle"
    FontAttributes="Bold" />
    <CarouselView Grid.Row="2" ItemsSource="{Binding Favorites}"
    PeekAreaInsets="0,0,40,0" IndicatorView="Indicator">
        <CarouselView.ItemsLayout>
            <LinearItemsLayout Orientation="Horizontal"

```

```

SnapPointsAlignment="Start" SnapPointsType="MandatorySingle" />
</CarouselView.ItemsLayout>
<CarouselView.EmptyViewTemplate>
  <DataTemplate>
    <Label Text="No favorites selected" />
  </DataTemplate>
</CarouselView.EmptyViewTemplate>
<CarouselView.ItemTemplate>
  <DataTemplate x:DataType="models:Photo">
    <Border Grid.RowSpan="2" StrokeShape="RoundRectangle
15,15,15,15" Padding="0" Margin="0,0,0,0" BackgroundColor="#667788" >
      <Image Source="{Binding Bytes, Converter={StaticResource
ToImage}}" Aspect="AspectFill" />
    </Border>
  </DataTemplate>
</CarouselView.ItemTemplate>
</CarouselView>
<IndicatorView Grid.Row="3" x:Name="Indicator" HorizontalOptions="Center" SelectedIndicatorColor
</Grid>

```

For the **Favorites** photos, again, **CarouselView** is used with a few changes from **CarouselView** displaying the **Recent** photos. The most visible change is that the **ItemsLayout** property is now using **MandatorySingle** for the value of **SnapPointsType**. This forces a behavior that only allows the user to swipe one image at a time, snapping each image into view.

The **ItemTemplate** property has also been changed to add a rounded border around each image, with a background color.

New to this **CarouselView** is the **EmptyViewTemplate** property. This is used to display the text "No favorites selected" when the **Favorites** property is empty.

Finally, **IndicatorView** was added to provide the user with a visual cue of how many items are in **CarouselView** and which item is currently displayed. **CarouselView** is connected to **IndicatorView** by the **IndicatorView** property of **CarouselView**. The **IndicatorView** property is set to the **x:Name** property of **IndicatorView**. The **IndicatorView** displays on the page as a series of horizontal light gray dots, with the dot representing the current image in red.

That is all—now, we can run the app and see both the most recent photos and the photos that have been marked as favorites.

Summary

In this chapter, we focused on photos. We learned how to import photos from the platform-specific photo galleries and how we can display them as a grid using **CollectionView** and in carousels using **CarouselView**. This makes it possible for us to build other apps and provides multiple options for presenting data to users, as we can now pick the best method for the situation.

Additionally, we learned about permissions and how to check and request permission to use protected resources in our app.

If you are interested in extending the app even further, try creating a page to view the details of the photo, or to view the photo in full screen by tapping on the photo.

In the next chapter, we will build an app using location services and look at how to visualize location data on a map.